0

Implementation notes

Naive-2 loads all N/p rows of A and N/p columns of B (full length N) into shared memory before the multiplication. A block of $BR \times BC$ threads needs $(BR + BC) \cdot (N/p) \cdot N \cdot 2$ bytes, which exceeds the GPU's per-block limit for most configurations at $N \geq 512$ (e.g. 512 KB at $N = 1024$, $p = 8$); those configurations are skipped and listed below.

Block MM launches p^2 threads in total, with each thread owning the same $T \times T$ block of C ($T = N/p$), worked through in 4×4 register sub-tiles (2×2 when $T = 2$). For each sub-tile, threads march across k in chunks of 32, cooperatively staging the needed A rows and B columns into shared memory (coalesced, padded by one column), synchronizing, then accumulating from shared memory only. C is written once at the end; global traffic per output drops from $2N$ (Naive-1) to $(BR + BC)\, N/(BR \cdot BC \cdot 4)$ for a $BR \times BC$ thread block.

The sweep is reduced to $p \leq 32$ and $N \leq 1024$, as discussed in class on Oct 15.

Naive-2 configurations skipped (shared memory needed vs. limit per block):

N	p	threadNum skipped	needed	limit
512	8	1, 2, 4, 8, 16, 32, 64	128–1024 KB	64 KB
512	16	2, 4, 8, 16, 32, 64, 128, 256	96–1024 KB	64 KB
512	32	4, 8, 16, 32, 64, 128, 256, 512, 1024	80–1024 KB	64 KB
1024	8	1, 2, 4, 8, 16, 32, 64	512–4096 KB	64 KB
1024	16	1, 2, 4, 8, 16, 32, 64, 128, 256	256–4096 KB	64 KB
1024	32	1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024	128–4096 KB	64 KB

Setup

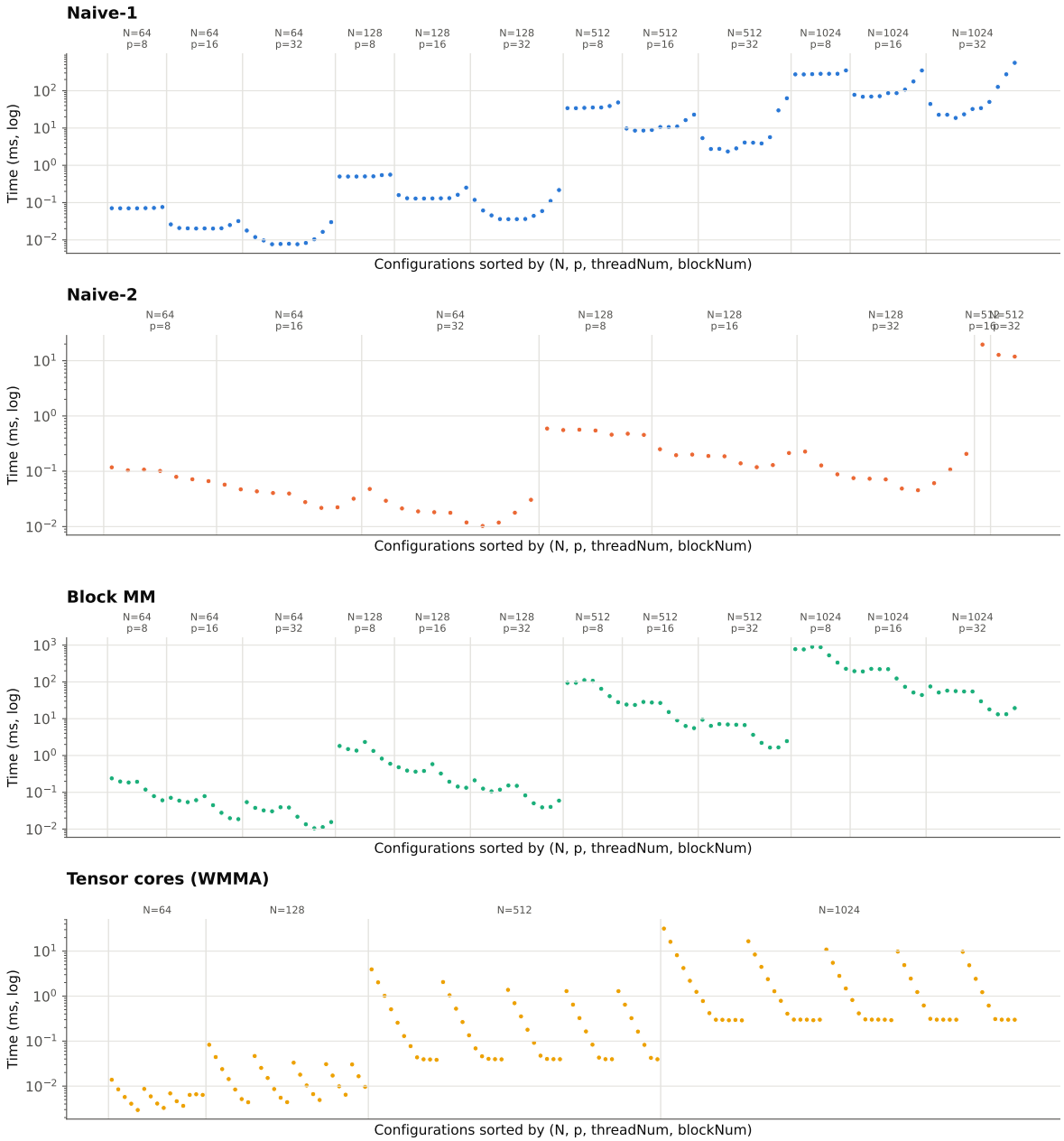
A and B are $N \times N$ row-major `half` matrices, C is `float`, with entries drawn uniformly from $[-1, 1]$. The sweep covers $N \in \{64, 128, 512, 1024\}$ and $p \in \{8, 16, 32\}$, with every power-of-two split of p^2 threads into blockNum $\times$ threadNum for Naive-1, Naive-2, and Block MM. Timing uses CUDA events on the kernel only (no host↔device copies), with one warm-up and the mean of five runs (one run if the kernel exceeds 200 ms). Every configuration is verified against a CPU float reference; all CUDA-core and tensor-core runs pass with max relative error $< 10^{-3}$.

Implementations

Kernel	Work per thread	Memory
Naive-1	$T \times T$ outputs, $T = N/p$	global only
Naive-2	same as Naive-1	all N/p rows of A / columns of B loaded into shared memory first
Block MM	same as Naive-1, computed in 4×4 register sub-tiles	A rows / B columns staged in shared memory in k-chunks of 32, reused across the block
WMMA	16×16 output tile per warp	tensor-core fragments
cuBLAS	`cublasHgemm`	library-chosen

1

1.1



Within each group, points run from fewest to most threads per block. So 1 thread per block then 2 then 4 then … p^2 threads per block. WMMA does not use p, so it is grouped by N

only.

1.2

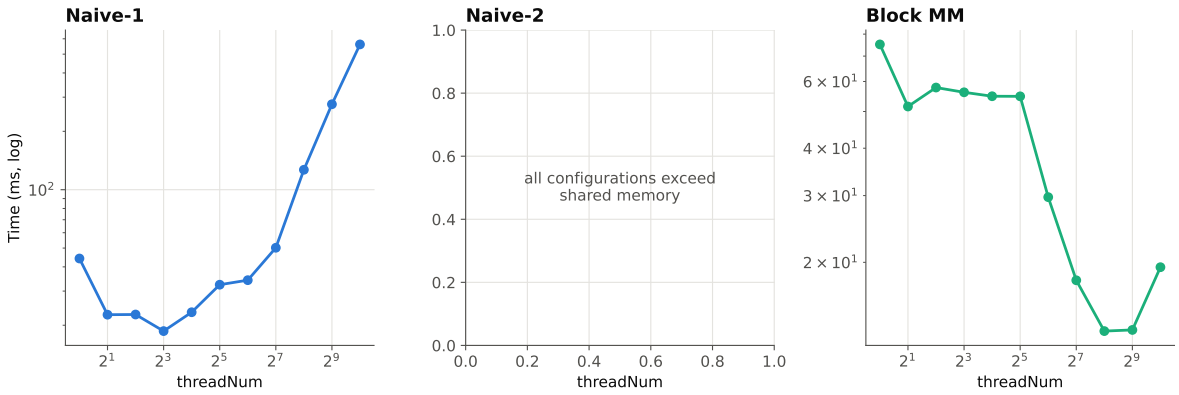
Implementation	N	p=8 (ms)	p=16 (ms)	p=32 (ms)	speedup p=8→32
Naive-1	64	0.070	0.020	0.008	9.2×
Naive-1	128	0.501	0.129	0.036	13.9×
Naive-1	512	34.054	8.552	2.354	14.5×
Naive-1	1024	274.071	69.006	18.658	14.7×
Naive-2	64	0.066	0.022	0.010	6.5×
Naive-2	128	0.455	0.119	0.045	10.0×
Naive-2	512	–	19.580	11.865	–
Naive-2	1024	–	–	–	–
Block MM	64	0.061	0.019	0.011	5.7×
Block MM	128	0.597	0.133	0.039	15.3×
Block MM	512	28.093	5.543	1.643	17.1×
Block MM	1024	227.090	44.002	13.171	17.2×

All three kernels improve with p: every doubling gives 4× the threads, so we expect $\approx 16\times$ speedup overall, though not all implementations fully reach it.

For Naive-1, each doubling of p cuts time by about 3.6–4× (about 14–15× from $p = 8$ to 32 at $N \geq 128$), close to the thread-count ratio since nothing else changes. Naive-2 scales similarly at $N \leq 128$ (about 6–10×), but at $N = 512$ only 1–2-thread blocks fit in shared memory, so $p = 16 \rightarrow 32$ gains only 1.7×, and $N = 1024$ cannot run at all. Block MM gains more than 4× from $p = 8$ to 16 (about 5× at $N \geq 512$) and about 3.3× from 16 to 32, for about 17× overall at $N \geq 512$ versus about 14.5× for Naive-1. Larger p widens the tile, so the best block grows from 64 to 256 threads and more threads share each staged tile. Thus adding both parallelism and data reuse. From $p = 16$ to 32 the best block size stays at 256 threads, so the extra gain is mainly from the additional threads.

1.3

Shown for the largest N and p; the other (N, p) pairs follow the same pattern.

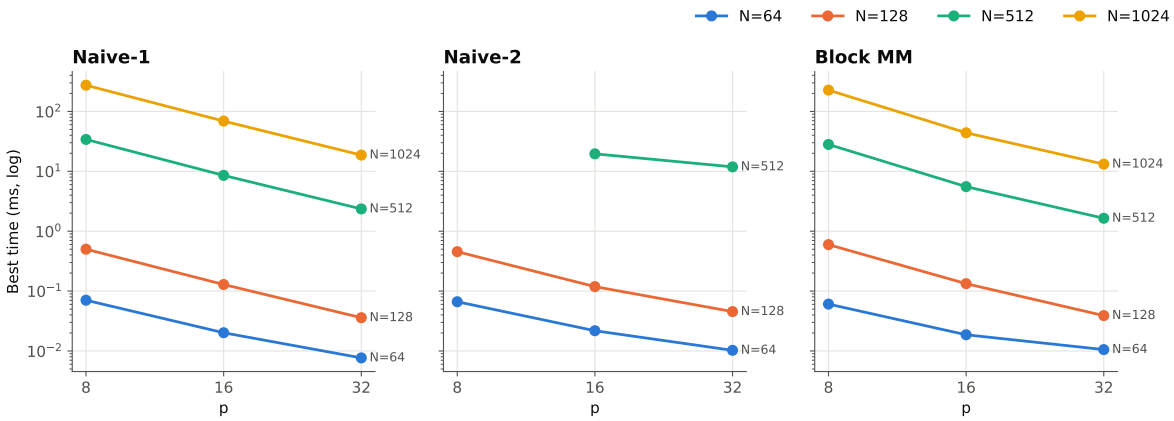


For the naive kernels ($blockNum \times threadNum = p^2$), many small blocks win. A block is confined to one SM, so 1 block $\times$ 1024 threads uses only one SM while 128 $\times$ 8 spreads across the device. Large blocks are also uncoalesced (see Part 2), which steepens the right end of the curve. Block MM is best with one block per tile and 64–256 threads; fewer blocks than tiles roughly doubles time per halving, and 1024 threads per block is slightly slower due to the register cap of 64 per thread (`__launch_bounds__(1024)`) limiting occupancy. WMMA time falls until there is one warp per 16 $\times$ 16 tile, then plateaus.

1.4

Implementation	N	p	blockNum	threadNum	Time (ms)	GFLOP/s
Block MM	64	8	1	64	0.0606	8.6
Block MM	64	16	1	256	0.0186	28.1
Block MM	64	32	4	256	0.0105	49.7
Block MM	128	8	1	64	0.5969	7.0
Block MM	128	16	1	256	0.1331	31.5
Block MM	128	32	4	256	0.0390	107.6
Block MM	512	8	1	64	28.0931	9.6
Block MM	512	16	1	256	5.5430	48.4
Block MM	512	32	4	256	1.6433	163.3
Block MM	1024	8	1	64	227.0904	9.5
Block MM	1024	16	1	256	44.0018	48.8
Block MM	1024	32	4	256	13.1712	163.0
Naive-1	64	8	16	4	0.0704	7.4
Naive-1	64	16	8	32	0.0202	26.0
Naive-1	64	32	128	8	0.0077	68.4
Naive-1	128	8	32	2	0.5010	8.4
Naive-1	128	16	64	4	0.1288	32.6
Naive-1	128	32	64	16	0.0360	116.7

Implementation	N	p	blockNum	threadNum	Time (ms)	GFLOP/s
Naive-1	512	8	32	2	34.0538	7.9
Naive-1	512	16	128	2	8.5520	31.4
Naive-1	512	32	128	8	2.3543	114.0
Naive-1	1024	8	32	2	274.0706	7.8
Naive-1	1024	16	128	2	69.0057	31.1
Naive-1	1024	32	128	8	18.6585	115.1
Naive-2	64	8	1	64	0.0664	7.9
Naive-2	64	16	4	64	0.0218	24.1
Naive-2	64	32	8	128	0.0103	51.1
Naive-2	128	8	1	64	0.4552	9.2
Naive-2	128	16	4	64	0.1188	35.3
Naive-2	128	32	8	128	0.0455	92.3
Naive-2	512	16	256	1	19.5805	13.7
Naive-2	512	32	512	2	11.8647	22.6



Every line slopes down meaning more threads (naive) or more tile reuse (Block MM) always helps in this range. Lines for successive N are separated by roughly $(N_2/N_1)^3$, as expected for $O(N^3)$ work; the 512→1024 gap is about 8×. Block MM lines sit 1–2 orders of magnitude below the naive kernels at large N and are flatter, because shared-memory tiling rather than thread count is the limiting factor.

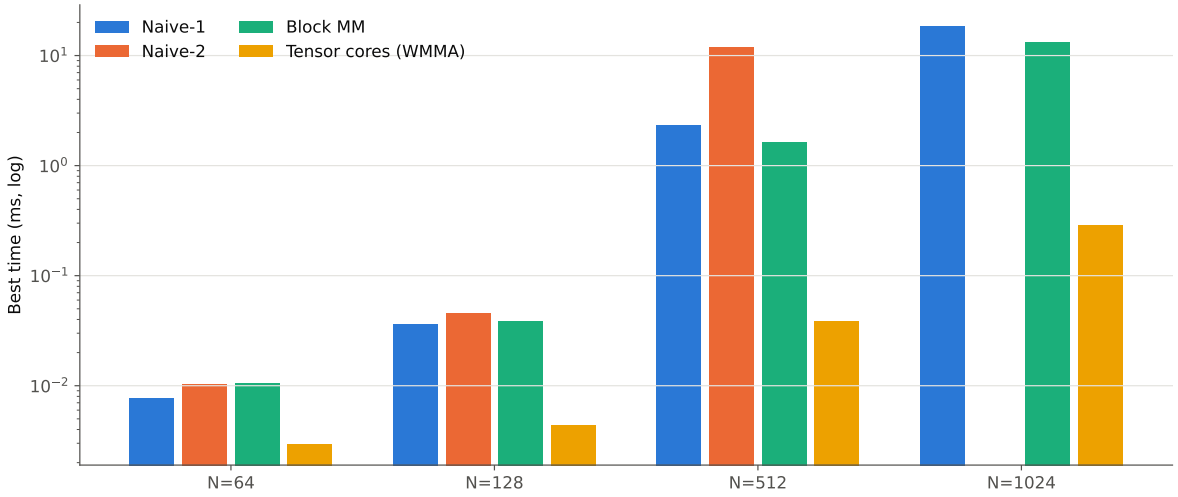
Tensor cores: best configuration per N

N	blockNum	threadNum	Warps	16×16 tiles	Time (ms)	GFLOP/s
64	16	32	16	16	0.0029	178
128	64	32	64	64	0.0044	958

N	blockNum	threadNum	Warps	16×16 tiles	Time (ms)	GFLOP/s
512	1024	32	1024	1024	0.0383	7002
1024	4096	32	4096	4096	0.2892	7424

The kernel uses `nvcuda::wmma` with $16 \times 16 \times 16$ half fragments and a float accumulator, with each warp grid-striding over output tiles. The best configurations launch about one warp per output tile; beyond that, extra warps have no work and time stays flat.

Tensor cores vs CUDA-core implementations



Tensor cores are fastest at every N; the gap over the best CUDA-core kernel (Block MM) grows with N as more of the time becomes compute.

cuBLAS comparison

N	Best own implementation	Own time (ms)	cuBLAS time (ms)	Performance ratio (own / cuBLAS)	cuBLAS GFLOP/s
64	Tensor cores (WMMA)	0.0029	0.0090	3.059	58
128	Tensor cores (WMMA)	0.0044	0.0086	1.953	491
512	Tensor cores (WMMA)	0.0383	0.0145	0.377	18,575

N	Best own implementation	Own time (ms)	cuBLAS time (ms)	Performance ratio (own / cuBLAS)	cuBLAS GFLOP/s
1024	Tensor cores (WMMA)	0.2892	0.0291	0.101	73,827

Performance ratio = own GFLOP/s ÷ cuBLAS GFLOP/s (= cuBLAS time ÷ own time); values above 1 mean our kernel is faster.

N	Naive-1 / cuBLAS	Naive-2 / cuBLAS	Block MM / cuBLAS	Tensor cores (WMMA) / cuBLAS
64	1.1754	0.8783	0.8538	3.0588
128	0.2378	0.1881	0.2193	1.9529
512	0.0061	0.0012	0.0088	0.3770
1024	0.0016	–	0.0022	0.1006

cuBLAS: interpretation

`cublasHgemm` is column-major, so row-major $C = AB$ is computed as column-major $C^T = B^T A^T$ by passing B before A so no explicit transposes needed. At $N \leq 128$ our WMMA kernel beats cuBLAS, since cuBLAS's fixed launch and heuristic overhead ($\approx 9\,\mu$s) dominates at small sizes. At large N cuBLAS pulls far ahead: it stages tiles in shared memory, pipelines loads with compute, and uses wider warp tiles, whereas our WMMA kernel loads fragments directly from global memory.

`cublasHgemm` accumulates and stores in FP16, so it differs from the FP32 reference by up to about 2% ($N = 512$) and 9% ($N = 1024$). Our kernels accumulate in FP32, which is more accurate but does more work per FLOP. A `cublasGemmEx` call with FP32 compute would be the like-for-like baseline.

Memory coalescing: request counts

From class (L10): requests = $\lceil R/32 \rceil$, where R is the range of data elements fetched across all threads of the warp, first to last. This is applied to every warp load and store of each kernel and summed. The table uses threadNum = $\min(p^2, 256)$ for the naive kernels and p^2 for Block MM.

N	p	Naive-1	Naive-2	Block MM	Naive-1 coalesced
64	8	417,792 (25.50)	256 (1.00)	14,336 (7.00)	16,384 (1.00)
64	16	90,112 (5.50)	256 (1.00)	3,072 (3.00)	16,384 (1.00)
64	32	24,576 (1.50)	640 (1.00)	512 (1.00)	16,384 (1.00)
128	8	12,910,592 (98.50)	1,024 (1.00)	212,992 (13.00)	131,072 (1.00)
128	16	2,424,832 (18.50)	1,024 (1.00)	40,960 (5.00)	131,072 (1.00)
128	32	327,680 (2.50)	2,560 (1.00)	4,096 (1.00)	131,072 (1.00)
512	8	12,952,010,752 (1544.00)	379,346,944 (2723.94)	51,380,224 (49.00)	8,388,608 (1.00)
512	16	2,218,786,816 (264.50)	64,897,024 (466.00)	8,912,896 (17.00)	8,388,608 (1.00)
512	32	71,303,168 (8.50)	2,007,040 (12.25)	262,144 (1.00)	8,388,608 (1.00)
1024	8	413,323,493,376 (6159.00)	25,428,131,840 (11938.54)	813,694,976 (97.00)	67,108,864 (1.00)
1024	16	69,793,218,560 (1040.00)	4,292,968,448 (2015.55)	138,412,032 (33.00)	67,108,864 (1.00)
1024	32	1,107,296,256 (16.50)	67,272,704 (30.19)	2,097,152 (1.00)	67,108,864 (1.00)

Load requests (requests per warp load instruction in brackets). The full table for every threadNum, including stores, is in `PA1/results_part2_requests.csv`; its `*_seg` columns give the hardware count (distinct 32-byte segments) for comparison.

2

Memory coalescing: why the counts differ

For Naive-1, neighbouring threads (consecutive t_x) read `B[k][tx*T + j]`, T elements apart. At $p = 32$ a warp is one row of 32 t_x values, so $R = 31T + 1$ and B needs about T requests (32 at $N = 1024$), while all lanes share t_y and read one A element (1 request): 16.5 per load on average. When $p < 32$ a warp covers $32/p$ values of t_y whose A rows are $T \cdot N$ elements apart, so R spans whole rows and the equation gives thousands of requests (6,159 at $N = 1024$, $p = 8$). Hardware only fetches the segments actually touched (6 per load there, `*_seg` columns); the equation counts the gaps too.

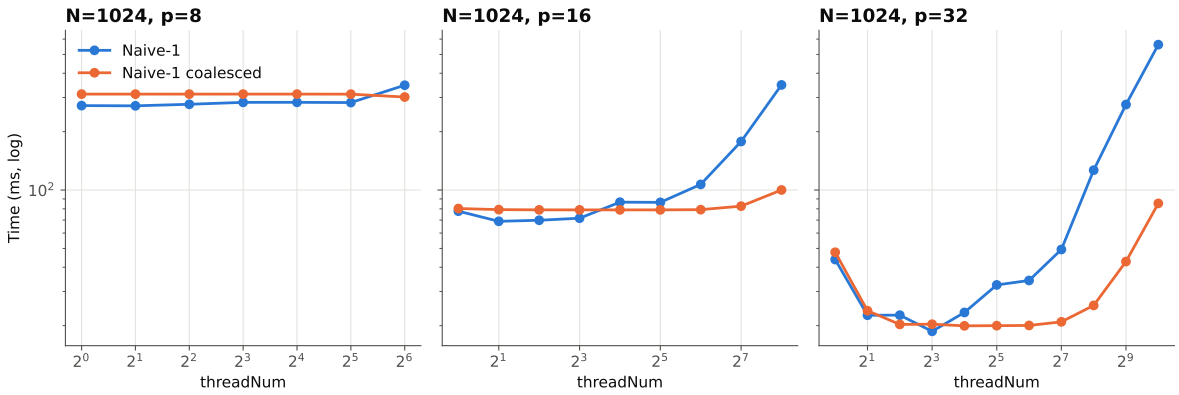
For Naive-2, staging loads are coalesced since consecutive threads read consecutive elements (1 request per load at $N \leq 128$). When rows do not fit in shared memory the kernel skips those configurations, so the counts for skipped cases reflect the chunked model used for counting only.

For Block MM, at $p = 32$ a warp reads one 32-element tile row (1 request). At $p < 32$ a warp covers $32/p$ tile rows that are N elements apart, so the equation gives $\lceil((32/p - 1)N + p)/32\rceil$: 97 at $N = 1024$, $p = 8$ and 33 at $p = 16$.

For Coalesced Naive-1, the data layout is changed, not the work assignment: - `Bp[(k·T + j)·p + tx] = B[k][tx·T + j]` - `Ap[(i·N + k)·p + ty] = A[ty·T + i][k]`

For fixed (i, j, k): consecutive threads (consecutive t_x) read consecutive halves of B', giving 1 request per load. For A', threads sharing the same t_y broadcast the same element; threads with consecutive t_y read consecutive halves and a warp reads only $\lceil 32/p \rceil$ distinct A' elements, also 1 request per load. Coalesced Naive-1 therefore needs exactly 1 request per load at every N and p.

Memory coalescing: timing



N	p	Max speedup at same config	Naive-1 best (ms)	Coalesced best (ms)
512	8	1.16× (threadNum=64)	34.16	37.67
512	16	1.83× (threadNum=256)	8.56	9.81
512	32	5.95× (threadNum=1024)	2.36	2.47
1024	8	1.15× (threadNum=64)	271.72	301.64

N	p	Max speedup at same config	Naive-1 best (ms)	Coalesced best (ms)
1024	16	3.48× (threadNum=256)	68.99	79.02
1024	32	6.58× (threadNum=1024)	18.69	19.95

Coalescing pays off once warps are full (threadNum 32): up to about 6.6× at $N = 1024$, $p = 32$. With fewer than 32 threads per block a warp is partly empty, so there is nothing to coalesce and the extra index arithmetic costs about 10%.